THIẾT KẾ VÀ TRIỂN KHAI M5: NOSQL, SERVING VÀ DASHBOARD TRONG HỆ THỐNG PHÂN TÍCH HÀNG KHÔNG

Tài liệu học tập theo các nhiệm vụ thành phần — Dự án AERO

Đối chiếu mã nguồn và bằng chứng triển khai ngày 07/11/2026

Tóm tắt

M5 là lớp phục vụ dữ liệu trong dự án phân tích hàng không, có nhiệm vụ chuyển kết quả xử lý từ các pipeline batch và streaming thành dữ liệu có thể truy vấn và trực quan hóa. Tài liệu trình bày M5 theo mười nhiệm vụ thành phần, từ phân tích đầu vào, thiết kế MongoDB, chuẩn bị hạ tầng, xây dựng chương trình nạp, phát triển truy vấn và dashboard, đến tích hợp streaming, kiểm thử, vận hành và định hướng Kubernetes. Mỗi nhiệm vụ được giải thích theo mục tiêu, cơ sở thiết kế, phương pháp thực hiện và kết quả cần đạt. Nội dung kết hợp mô tả lý thuyết với cách tổ chức mã nguồn hiện có, giúp thành viên hiểu được nguyên nhân của các quyết định kỹ thuật thay vì chỉ sao chép lệnh. Phạm vi đã kiểm thử là luồng dữ liệu lịch sử M2 vào MongoDB và dashboard trên Docker local; streaming còn chờ M4 tích hợp, còn Kubernetes là phương án triển khai tiếp theo.

Từ khóa: serving layer; MongoDB; data contract; batch ingestion; dashboard; streaming; Docker; Kubernetes.

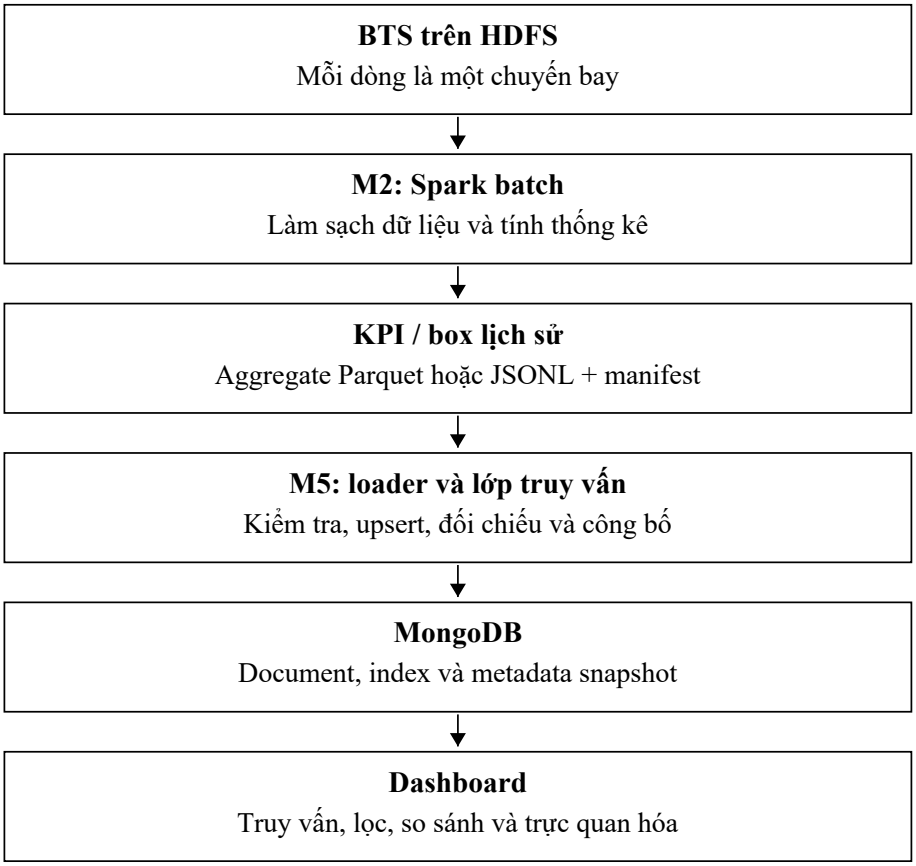
1. Bối cảnh, mục tiêu và cách tổ chức nhiệm vụ M5

Trong một hệ thống dữ liệu lớn, lưu được dữ liệu và xử lý được dữ liệu chưa đồng nghĩa với việc người dùng có thể sử dụng kết quả. Dữ liệu chi tiết trong HDFS phù hợp với công việc xử lý và phân tích bằng Spark, nhưng người xem dashboard thường cần những câu trả lời cụ thể: có bao nhiêu chuyến bay, tỷ lệ đúng giờ thay đổi thế nào, hãng nào bị trễ nhiều, hoặc tuyến nào có lưu lượng cao. M5 tạo ra lớp trung gian giữa kết quả tính toán và nhu cầu truy vấn của ứng dụng.

Theo phân công dự án, M1 phụ trách ingestion và HDFS, M2 phụ trách Spark batch, M3 phụ trách nguồn API hoặc mô phỏng và Kafka, M4 phụ trách Spark Structured Streaming. M5 tiếp nhận KPI lịch sử từ M2 và KPI streaming từ M4 để tổ chức trong MongoDB và hiển thị trên dashboard [1]. Vì vậy, M5 không chịu trách nhiệm làm sạch lại toàn bộ dữ liệu BTS hoặc thực hiện lại các phép thống kê mà M2 đã hoàn tất. Trách nhiệm của M5 là bảo đảm kết quả được nhận đúng, lưu đúng, truy vấn đúng và trình bày đúng.

Mục tiêu tổng quát của M5 gồm tính chính xác của dữ liệu phục vụ, khả năng truy xuất nguồn, khả năng nạp lại và khả năng tái lập môi trường. Một dashboard đẹp nhưng đọc sai snapshot hoặc gộp sai mẫu số chưa đạt yêu cầu. Ngược lại, một database đúng nhưng không có truy vấn và cách diễn giải phù hợp cũng chưa hoàn thành chức năng phục vụ

thông tin. Sản phẩm M5 phải kết hợp cả lớp lưu trữ, lớp truy vấn, giao diện và bằng chứng nghiệm thu.



Hình 1. Luồng lịch sử từ BTS trên HDFS qua M2 đến MongoDB và dashboard M5. Sơ đồ được vẽ lại theo các hình đã cung cấp trong hội thoại; nhánh bàn giao JSONL là cách chuyển cùng các bảng KPI sang M5 khi không đọc HDFS trực tiếp.

Tài liệu tổ chức M5 thành mười nhiệm vụ lớn. Sáu nhiệm vụ đầu tạo thành luồng batch phục vụ lịch sử; nhiệm vụ thứ bảy xác định cách nhận streaming; nhiệm vụ thứ tám kiểm chứng toàn bộ phần phục vụ; nhiệm vụ thứ chín tổ chức vận hành và bàn giao; nhiệm vụ thứ mười mở rộng hạ tầng lên Kubernetes. Đây là thứ tự phụ thuộc về thiết kế và triển khai, không phải yêu cầu mỗi nhiệm vụ phải nằm trên một trang riêng.

2. Task 1 — Phân tích đầu vào và xây dựng hợp đồng dữ liệu

2.1. Mục tiêu và cơ sở thiết kế

Nhiệm vụ đầu tiên là xác định M5 nhận dữ liệu gì, một dòng dữ liệu đại diện cho điều gì và các trường có ý nghĩa như thế nào. Nội dung này được mô tả bằng hợp đồng dữ liệu, hay data contract. Hợp đồng bao gồm schema, kiểu dữ liệu, khóa nghiệp vụ, đơn vị đo, phạm vi thời gian, đường dẫn bàn giao và quy tắc cập nhật. Nếu thiếu hợp đồng, hai chương trình vẫn có thể trao đổi được file nhưng hiểu khác nhau về cùng một giá trị.

Khái niệm quan trọng nhất là mức dữ liệu, thường gọi là grain. Trong *kpi_month_overall*, một document biểu diễn một tháng của một năm; trong *kpi_year_route*, một document biểu diễn một tuyến đi–đến trong một năm. Đây đều là dữ liệu tổng hợp, không phải các chuyến bay riêng lẻ. Vì vậy, số document trong MongoDB không cần bằng số chuyến bay trong dữ liệu gốc.

2.2. Phương pháp thực hiện trong dự án

Đầu vào lịch sử hiện tại gồm 13 bảng KPI và 7 bảng phân phối box. M2 bàn giao các bảng dưới dạng aggregate Parquet trên HDFS hoặc bộ export gồm JSONL, CSV và *manifest.json*. Manifest mô tả bảng, schema Spark, các khóa, số dòng và coverage. Loader M5 ưu tiên JSONL vì định dạng này giữ được số, boolean và null; CSV được dùng để đọc và đối chiếu thuận tiện [2].

Với snapshot *20261107T042703414867Z*, bộ dữ liệu có 64.249 dòng KPI và 10.672 dòng box, tổng cộng 74.921 document dữ liệu. Coverage gồm 51 tháng từ tháng 1/2022 đến tháng 3/2026. Tổng trường flights trong bảng KPI tháng là 29.328.846 chuyến sau cleaning. Những số này mô tả bộ tham chiếu hiện tại, không phải định mức bất biến cho mọi lần export. Các document quản lý lần nạp được tính riêng.

M5 cần thống nhất cả các quy ước nghiệp vụ. Tỷ lệ được lưu trong khoảng 0–1 và chỉ nhân 100 khi hiển thị phần trăm. Delay tính bằng phút, có thể âm khi chuyến đến sớm. Null biểu thị thiếu hoặc không xác định giá trị, không tự động đồng nghĩa với số 0. Các bảng box cần phân biệt loại delay *arr* hoặc *dep* và quần thể *all* hoặc *late*; thiếu các chiều này sẽ làm lẫn các phân phối khác nhau.

Đối với streaming, hợp đồng phải bổ sung cửa sổ thời gian, nguồn, phiên bản cập nhật và ngữ nghĩa ghi. M4 phải xác định record là toàn bộ aggregate hiện tại của cửa sổ hay chỉ là phần tăng thêm. Writer M5 hiện nhận toàn bộ aggregate, nên một record delta không thể đưa vào cùng cách ghi mà không thiết kế lại adapter [3].

2.3. Kết quả cần đạt

Task này hoàn thành khi M2 và M5 có thể chỉ ra cùng một bảng, giải thích giống nhau về một dòng, một khóa và một KPI. Đầu ra là manifest hợp lệ, bộ file đầy đủ và tài liệu quy ước. Với M4, đầu ra là contract được hai bên xác nhận hoặc danh sách phần adapter cần sửa. Trong mã nguồn, *m5/contracts.py* hiện thực các quy tắc kiểm tra; tài liệu bàn giao M2 và contract M4 là căn cứ để bảo trì các quy tắc đó.

3. Task 2 — Thiết kế mô hình document và chỉ mục MongoDB

3.1. Từ bảng tổng hợp đến document truy vấn

MongoDB lưu dữ liệu dưới dạng document có trường và giá trị; collection là tập document được tổ chức cho cùng một mục đích. Trong M5, thiết kế chủ động giữ cấu trúc gần với output M2 để giảm biến đổi không cần thiết và làm rõ quá trình đối chiếu. Dữ liệu phân tích chi tiết tiếp tục nằm ở HDFS; MongoDB lưu bản tổng hợp để ứng dụng truy cập theo các câu hỏi đã xác định.

Thiết kế cơ sở dữ liệu không chỉ là chọn tên collection. M5 phải xác định điều kiện duy nhất của một document, những chiều thường được lọc và cách phân biệt các bộ dữ liệu khác nhau. Nếu bỏ qua vấn đề phiên bản, lần nạp mới có thể trộn với dữ liệu cũ, khiến dashboard đồng thời đọc các bảng thuộc nhiều lần export.

3.2. Khóa, snapshot và metadata

M5 bổ sung trường *snapshot_id* vào document lịch sử. Giá trị này được tính từ nội dung và schema của bộ dữ liệu, thay vì lấy trực tiếp tên thư mục export. Ví dụ, khóa duy nhất của KPI tháng là tổ hợp *snapshot_id*, *year* và *month*. Khóa của KPI tuyến cần thêm *origin* và *dest*; khóa box cần chứa đầy đủ *var* và *population*. Unique compound index bảo đảm một tổ hợp khóa chỉ xuất hiện tối đa một lần trong collection [6].

Ngoài 13 collection *kpi_ và 7 collection box_*, hệ thống có metadata. *batch_runs* lưu manifest, digest, nguồn, trạng thái và báo cáo của từng bộ nạp. *serving_state* lưu con trỏ tới snapshot lịch sử đang được phục vụ. *serving_locks* lưu lease điều phối loader. Các collection này có chức năng vận hành, vì vậy không được cộng vào số document KPI/box của bộ dữ liệu.

Chỉ mục duy nhất có tên *snapshot_business_key* được tạo từ *snapshot_id* và khóa trong manifest. Một số collection có thêm index theo hãng, sân bay đi hoặc sân bay đến để hỗ trợ mẫu lọc tương ứng. Tuy nhiên, hiệu quả của index phải được đánh giá bằng truy vấn thực và explain; không thể suy ra rằng mọi phép lọc hoặc sắp xếp đều tối ưu chỉ vì collection đã có index.

3.3. Kết quả và tiêu chí hoàn thành

Đầu ra của task là danh mục collection, schema document, unique keys, các index hỗ trợ truy vấn và mô hình metadata nhất quán. Với bộ mẫu, MongoDB phải chứa đúng 74.921 document dữ liệu của snapshot đã chọn. Tổng document toàn database có thể lớn hơn khi có metadata hoặc nhiều snapshot. Khi kiểm tra, thành viên phải luôn biết mình đang đếm bộ nào thay vì chỉ nhìn tổng toàn collection.

Mã nguồn trọng tâm là *ensure_indexes()* và các hàm quản lý snapshot trong *m5/batch_writer.py*. Tài liệu *MONGODB_COLLECTIONS_QUERIES.md* ghi các mẫu

query và quy ước lưu trữ. Việc giữ nhiều snapshot tạo điều kiện truy vết, nhưng cũng đòi hỏi chính sách lưu giữ khi vận hành lâu dài; code hiện tại không tự xóa các snapshot cũ.

4. Task 3 — Chuẩn bị môi trường và triển khai hạ tầng serving

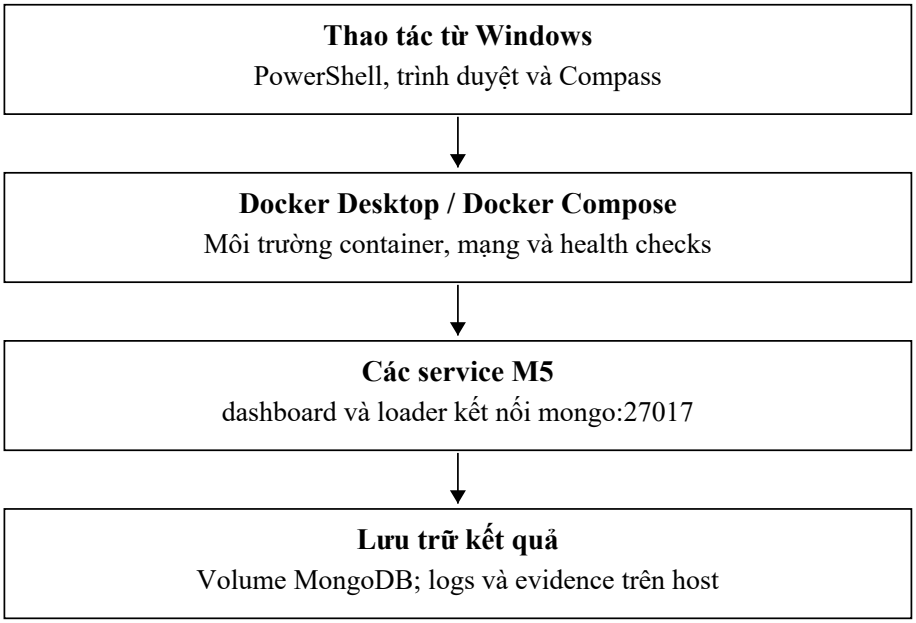
4.1. Vai trò của từng thành phần hạ tầng

Hạ tầng phục vụ phải cung cấp môi trường chạy ứng dụng, kết nối giữa các thành phần và lưu trữ bền vững. Phiên bản local của M5 sử dụng Docker Desktop để chạy Linux containers trên Windows, còn Docker Compose khai báo các service, mạng, volume và health check. Docker image đóng gói mã nguồn cùng dependencies; container là một phiên chạy từ image đó. Hai khái niệm này giúp phân biệt bản đóng gói phần mềm với tiến trình đang hoạt động.

MongoDB server chịu trách nhiệm lưu document và index. PyMongo là thư viện Python để loader và dashboard kết nối tới server này. Streamlit cung cấp ứng dụng web Python, còn Plotly xây dựng biểu đồ tương tác. Compass là công cụ xem database bằng giao diện, có thể cài thêm trên Windows; nó không thay cho MongoDB server. Với cách chạy chuẩn bằng Docker, thành viên không cần cài riêng Python hoặc MongoDB Server trên Windows.

4.2. Triển khai Docker local

compose.yml định nghĩa ba service: *mongo*, *dashboard* và *loader*. Hai service đầu chạy thường trực; loader được khởi tạo khi nạp hoặc xác minh rồi kết thúc. Script *start_m5.ps1* kiểm tra cấu hình, build image và khởi động MongoDB/dashboard, sau đó chờ health checks. Health check cho biết tiến trình sẵn sàng đáp ứng, chưa chứng minh dữ liệu KPI đã được nạp đúng.



Hình 2. Quan hệ giữa môi trường chạy M5, database và kết quả phục vụ. Các khối biểu diễn thành phần triển khai; chú thích mô tả trách nhiệm, không hàm ý Compose là một nguồn dữ liệu nghiệp vụ.

Trên Windows, trình duyệt truy cập dashboard tại *http://localhost:8501*; Compass kết nối *mongodb://localhost:27017*. Trong mạng container, dashboard và loader dùng *mongodb://mongo:27017*. Tên mongo là hostname của service trong mạng Compose, không phải địa chỉ dùng trực tiếp cho Compass Windows. Nếu cổng host đã bị chiếm, *.env* có thể đổi *M5_MONGO_PORT* hoặc *M5_DASHBOARD_PORT* mà không đổi cổng MongoDB nội bộ.

Dữ liệu MongoDB được giữ trong volume *aero-serving_m5_mongo_data*, mount vào */data/db*. Input được đưa vào loader bằng bind mount chỉ đọc; logs và evidence được mount ra thư mục dự án. Vì vậy, dừng container không đồng nghĩa xóa database, nhưng xóa volume sẽ ảnh hưởng dữ liệu. Việc phân biệt image, container và volume là điều kiện cần để thao tác hạ tầng đúng.

4.3. Kết quả cần đạt

Task này hoàn thành khi thành viên có thể dựng lại môi trường từ code, Docker đọc được cả Client và Server, MongoDB/dashboard healthy và địa chỉ kết nối đúng. Cấu hình hiện tại publish hai cổng trên loopback, phù hợp demo local. Triển khai dùng chung qua mạng cần quyết định thêm host, truy cập, xác thực và sao lưu. Một MongoDB instance hiện tại không được trình bày như cụm dự phòng nhiều máy.

5. Task 4 — Xây dựng pipeline nạp dữ liệu lịch sử

5.1. Mục tiêu của loader

Loader chuyển dữ liệu M2 thành document có thể truy vấn, đồng thời bảo vệ tính nhất quán của bộ phục vụ. Một lần nạp đạt yêu cầu phải đọc đúng nguồn, không tạo bản ghi trùng khi chạy lại, phát hiện lỗi trước khi công bố và cho phép phục hồi khi tiến trình gián đoạn. Tính chất nạp lại không làm thay đổi kết quả ngoài ý muốn thường được gọi là idempotency trong phạm vi thao tác đang xét.

Luồng bắt đầu ở *scripts/load_m2.ps1*. Script nhận đường dẫn snapshot Windows, kiểm tra có manifest, mount thư mục vào container rồi gọi *python -m m5.load_batch*. Điểm vào Python lựa chọn giữa chế độ chỉ kiểm tra và chế độ nạp thật. Cách tách này giúp thành viên kiểm tra input trước khi tác động tới database.

5.2. Kiểm tra, ghi theo lô và công bố snapshot

Ở chế độ CheckOnly, *inspect_snapshot()* đọc manifest và JSONL, kiểm tra đủ catalog, kiểu dữ liệu, khóa, số dòng, coverage và các điều kiện liên quan. Fingerprint được tính từ dữ liệu chuẩn hóa và schema. Kết quả PASS ở chế độ này xác nhận đầu vào đạt kiểm tra, chưa xác nhận đã có document trong MongoDB. Báo cáo được ghi ra *evidence/load_report.json*.

Khi nạp thật, writer giữ lease để điều phối các loader, tạo index và ghi bằng các thao tác *ReplaceOne* có upsert. Mỗi lô hiện gồm 1.000 thao tác. Bulk write gom nhiều thao tác trong một lời gọi thay vì gửi từng ghi riêng lẻ [7]. Trong mô hình của dự án, unique key và snapshot được dùng cùng nhau để kiểm soát tính nhất quán, không chỉ để giảm số bản ghi trùng.

Sau khi ghi, loader kiểm tra nguồn vẫn có fingerprint ban đầu rồi đọc lại dữ liệu MongoDB để so digest theo khóa. Chỉ khi các kiểm tra này đạt, snapshot mới được đánh dấu published và con trỏ phục vụ được chuyển sang bộ đó. Dashboard lấy con trỏ active và truy vấn theo snapshot tương ứng; do đó một lần nạp lỗi không thay bộ đang được người dùng xem.

Nếu bộ cùng nội dung đã published, loader đọc lại kiểm tra và trả trạng thái *ALREADY LOADED*, thay vì tạo thêm bộ dữ liệu. Nếu đây là snapshot cũ trong database có bộ mới hơn, nạp lại không tự động chuyển con trỏ phục vụ về quá khứ. Với lần nạp chưa hoàn tất, cơ chế metadata cho phép retry; logic khôi phục cũng xử lý ranh giới giữa verify và cập nhật con trỏ.

5.3. Nhánh đọc Parquet trên HDFS và đầu ra nhiệm vụ

Ngoài JSONL, *m5/load_parquet.py* dùng Spark đọc các bảng ở */aero/analytics/eda/kpi* và */aero/analytics/eda/box*, stage JSONL tạm rồi gọi cùng writer. Adapter không đọc lại toàn bộ cleaned flights ở */aero/processed*. Khi schema và giá trị trùng với bộ JSONL, hai nhánh có cùng fingerprint và được nhận diện là cùng nội dung. Nhánh này cần HDFS thực sự có bảng aggregate và mạng có thể truy cập NameNode/DataNode; các job ETL/EDA phải ổn định trong suốt lần đọc.

Đầu ra của task là snapshot MongoDB được công bố, log nạp và report chứa định danh, số document và coverage. Với bộ tham chiếu, một lần nạp đầu thành công có thông báo *LOAD COMPLETE* và 74.921 document; lần lặp lại có *ALREADY LOADED*. Đây là kết quả của pipeline chuyển/lưu dữ liệu, không phải bằng chứng độc lập rằng mọi công thức M2 đều đúng từ raw.

6. Task 5 — Xây dựng lớp truy vấn và tính KPI hiển thị

6.1. Truy vấn theo đúng mức dữ liệu

Lớp truy vấn chuyển bộ lọc của người dùng thành điều kiện đọc MongoDB. Tách lớp này khỏi giao diện giúp kiểm soát quy tắc nghiệp vụ tập trung, thay vì mỗi biểu đồ tự viết một query khác nhau. *m5/queries.py* xác định các collection hợp lệ, kiểm tra chiều lọc và luôn đưa *snapshot_id* vào điều kiện đọc.

Không phải bảng nào cũng hỗ trợ mọi chiều. KPI tuyến trong bộ hiện tại có năm, sân bay đi và sân bay đến, nên không thể cung cấp bộ lọc tháng chỉ bằng cách thêm một widget. Các bảng mùa vụ theo tháng trong năm hoặc giờ khởi hành đã gộp toàn kỳ cũng không tự có thêm chiều năm. Nếu cần góc nhìn mới, M2 và M5 phải thống nhất thêm bảng nguồn hoặc cách gộp hợp lệ.

6.2. Gộp KPI từ thống kê cộng được

Nhiều KPI là tỷ lệ hoặc trung bình, không thể cộng trực tiếp. Ví dụ, tỷ lệ đúng giờ trên quần thể chuyến đến hợp lệ được tính bằng tổng on_time chia tổng arr_n. Tỷ lệ hủy dùng tổng cancelled chia tổng flights. Khi người dùng chọn nhiều tháng, M5 phải cộng các tử số và mẫu số rồi tính lại, thay vì lấy trung bình các tỷ lệ đã có.

Giả sử tháng A có 10 chuyến hợp lệ và 9 chuyến đúng giờ, còn tháng B có 100 chuyến hợp lệ và 50 chuyến đúng giờ. Trung bình đơn giản của 90% và 50% là 70%, nhưng tỷ lệ đúng giờ của cả hai tháng là 59/110, xấp xỉ 53,6%. Ví dụ này cho thấy việc giữ count và sum trong KPI nguồn là cần thiết để ứng dụng gộp đúng.

m5/metrics.py hiện thực các phép chia có kiểm tra mẫu số, các trường cộng được và công thức chỉ số của nhóm. Khi thiếu mẫu số hoặc mẫu số bằng 0, kết quả phải biểu đạt

trạng thái không xác định phù hợp, không tự tạo tỷ lệ 0. Điểm *reliability_score* là thước đo do nhóm định nghĩa, không phải tiêu chuẩn ngành hàng không; khi dùng cần nêu công thức và giới hạn.

Các phân vị box như Q1, median và Q3 không có tính cộng. Trung bình median các tháng không cho median của cả năm. Vì vậy, giao diện chọn đúng bảng box mà M2 đã tính ở mức tương ứng. Các phép gộp và quần thể phải được xem như một phần của hợp đồng, không phải chi tiết định dạng biểu đồ.

6.3. Đầu ra và cách kiểm chứng

Task này tạo ra API nội bộ bằng các hàm Python để app truy vấn và tóm tắt dữ liệu nhất quán. Đầu ra cần có điều kiện lọc hợp lệ, kết quả đúng snapshot, thứ tự dữ liệu ổn định và phép gộp đúng. Kiểm thử cần bao gồm nhiều nhóm có kích thước khác nhau, dữ liệu thiếu, mẫu số bằng 0 và delay âm. Có thể đối chiếu một tháng cụ thể trong MongoDB với đúng hàng JSONL/CSV nguồn trước khi so toàn bộ dashboard.

7. Task 6 — Xây dựng dashboard và cách diễn giải kết quả

7.1. Từ câu hỏi phân tích đến trang hiển thị

Dashboard là sản phẩm người dùng trực tiếp tương tác, nhưng cấu trúc trang nên bắt đầu từ câu hỏi nghiệp vụ. Tổng quan trả lời quy mô chuyến và chất lượng vận hành; thời gian trả lời biến động và mùa vụ; hãng, sân bay và tuyến hỗ trợ so sánh nhóm; phân phối cho biết độ trải rộng và ngoại lệ của delay. Trang dữ liệu/vận hành giúp giải thích bộ nào đang được xem, còn trang streaming dành cho luồng mới khi được tích hợp.

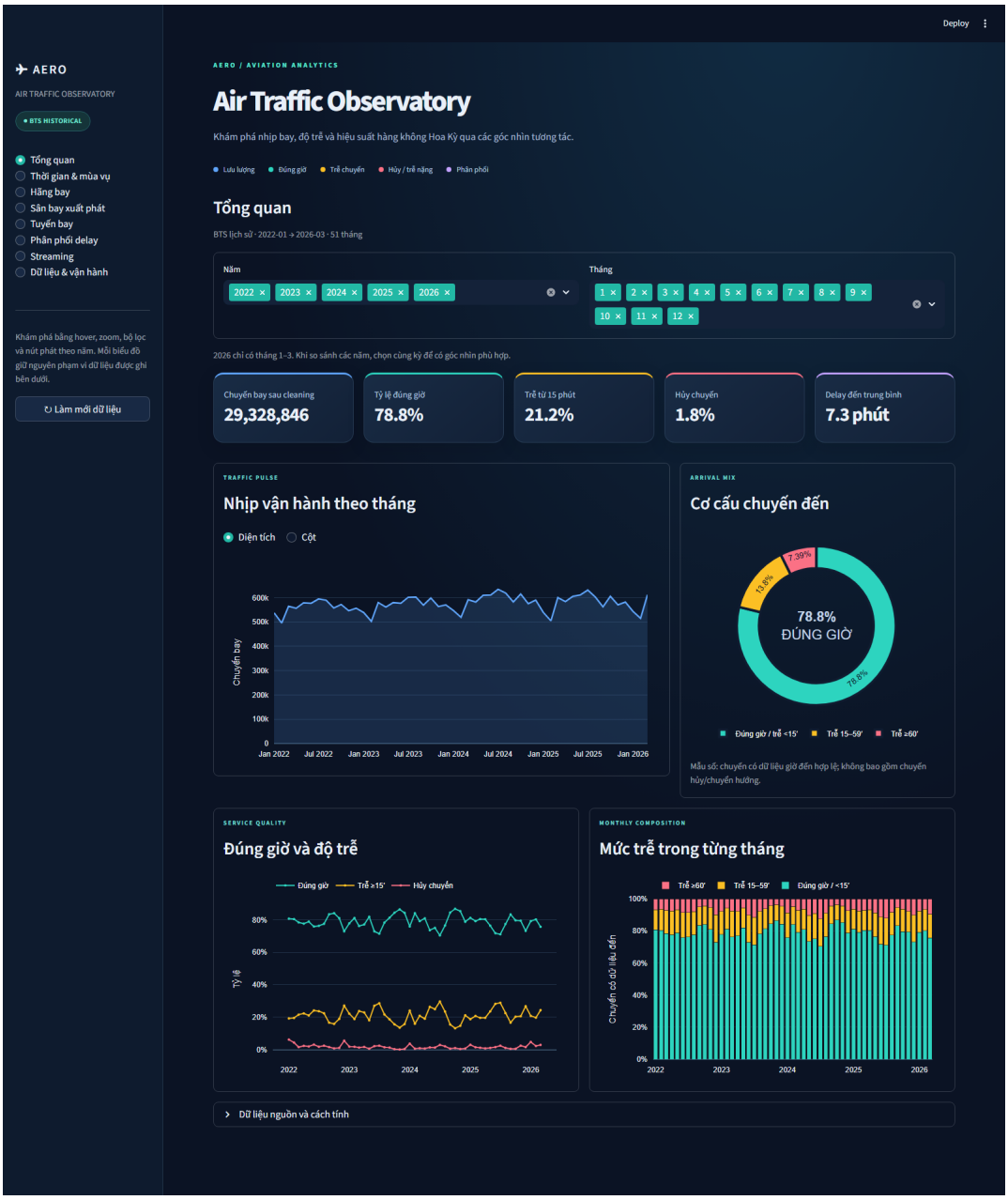
Ứng dụng hiện dùng Streamlit với tám trang: Tổng quan, Thời gian & mùa vụ, Hãng bay, Sân bay xuất phát, Tuyến bay, Phân phối delay, Streaming và Dữ liệu & vận hành. Backend Python truy vấn MongoDB qua các module dùng chung; frontend hiển thị trong trình duyệt. Kiến trúc client–server này giúp hiểu vì sao dữ liệu và code chạy trên host ứng dụng, còn người dùng chỉ mở giao diện [8].

7.2. Triển khai giao diện và lựa chọn biểu đồ

app.py tổ chức trang, widget và luồng nhận dữ liệu; *m5/charts.py* xây các biểu đồ Plotly; *assets/dashboard.css* định dạng giao diện. Việc phân tách giúp thay đổi hình thức mà vẫn giữ nguyên lớp truy vấn và phép tính. Cache được dùng cho kết nối hoặc dữ liệu đọc trong phạm vi cấu hình hiện tại; thao tác làm mới cho phép người dùng cập nhật sau ingest.

Lưu lượng theo thời gian phù hợp với đường, diện tích hoặc cột. Donut và cột chồng thể hiện cơ cấu trên cùng một quần thể không chồng lấn; các nhóm đúng giờ, trễ vừa và trễ nặng phải được tính để không đếm trùng. Bubble và treemap hỗ trợ so quy mô với chất

lượng theo hãng/sân bay, Sankey thể hiện các top tuyến đang chọn, còn box-plot dùng phân vị M2 có sẵn. Màu sắc cần nhất quán và có chú giải để người đọc không phải đoán ý nghĩa.



Hình 3. Ảnh dashboard thực tế của project, sử dụng snapshot BTS lịch sử từ 2022-01 đến 2026-03. Ảnh là evidence kiểm thử ngày 07/11/2026, không phải minh họa dữ liệu streaming.

Mỗi trang cần hiển thị phạm vi thời gian và các giới hạn quan trọng. Năm 2026 hiện chỉ có tháng 1–3, nên so tổng cả năm với các năm trước có thể dẫn đến diễn giải sai. Animation theo năm là lịch sử chứ không phải live. Sankey chỉ biểu diễn top tuyến đang được chọn, không đại diện toàn bộ mạng bay. Quần thể chuyến đến hợp lệ dùng cho OTP và cơ cấu chuyến đến cũng khác tổng chuyến theo lịch.

7.3. Kết quả cần đạt

Dashboard đạt yêu cầu khi người xem lọc được dữ liệu đúng chiều, đọc được ý nghĩa chỉ số và đối chiếu được kết quả với nguồn. Hover, chú giải, zoom, ảnh PNG và CSV là các công cụ hỗ trợ khám phá, không thay cho kiểm chứng số liệu. Đối với bản hiện tại, phải kiểm tra cả tám trang không có lỗi, bộ lọc có dữ liệu xử lý đúng và các trạng thái trống được giải thích rõ.

8. Task 7 — Thiết kế và tích hợp luồng streaming từ M4

8.1. Khác biệt so với batch

Batch thường bàn giao một bộ dữ liệu ổn định để nạp theo snapshot. Streaming có các cửa sổ đang thay đổi, sự kiện đến muộn và thao tác ghi được gửi lại sau lỗi. Vì vậy, chỉ dùng một lệnh ghi MongoDB chưa đủ; hai bên phải xác định khi nào một record là mới, khi nào là bản sửa và khi nào chỉ là replay. M5 cần bảo vệ giá trị mới trước các bản cập nhật cũ, đồng thời không làm số liệu tăng hai lần.

Writer hiện tại nhận aggregate đầy đủ của một cửa sổ. Khóa logic gồm source, pipeline_id, window_start, window_end và toàn bộ dimensions. Các trường bắt buộc được kiểm tra theo contract; thời gian có timezone và được chuẩn hóa UTC. M4 chịu trách nhiệm event time, watermark, checkpoint và cơ chế tạo revision bền vững qua restart. M5 không tự lấy thời điểm nhận làm thời điểm phát sinh sự kiện [3].

8.2. Phiên bản cập nhật, replay và độ mới

Khi cùng khóa và revision đã có cùng payload, writer bỏ qua replay. Revision thấp hơn không được ghi đè revision mới; revision mới thay toàn bộ aggregate. Hai payload khác nhau nhưng dùng cùng revision được xem là xung đột. Quy tắc này dựa trên phiên bản, khác với việc mỗi thông điệp tự cộng vào số đếm. Nếu M4 muốn gửi delta, cần thiết kế adapter và cơ chế dedup phù hợp trước khi sử dụng.

Độ mới cần được trình bày bằng các mốc có ý nghĩa. Window phản ánh phạm vi thời gian dữ liệu; ingested_at phản ánh khi M5 thật sự nhận bản cập nhật mới. Replay không được làm ingested_at thay đổi để dữ liệu cũ trông như vừa cập nhật. Dashboard hiện cấu hình refresh 10 giây và stale sau 60 giây, nhưng các ngưỡng phải điều chỉnh theo nhịp M4 và đo trong thử nghiệm tích hợp, không coi là SLA đã được chứng minh.

8.3. Trạng thái hiện tại và kết quả cần nghiệm thu

Đã có *m5/stream_writer.py*, contract đề xuất và kiểm thử version/replay guards. Tuy nhiên, chưa có bằng chứng M3, Kafka, M4 và M5 hoạt động end-to-end trong điều kiện thực tế của project. Nhiệm vụ tích hợp chỉ hoàn tất sau khi M4 xác nhận contract hoặc adapter được sửa theo output thật, rồi có thử nghiệm window mới, late update, replay, restart và trạng thái stale trên UI.

Nguồn *aviationstack* và *simulated* phải phản ánh đúng dữ liệu đang dùng. Không đưa fixture vào database phục vụ rồi gọi đó là luồng API live. Tương tự, không suy ra exactly-once toàn hệ thống chỉ từ checkpoint hoặc test riêng writer. Tính bảo đảm phải được đánh giá cùng source, state, sink, retry và kịch bản lỗi của pipeline.

9. Task 8 — Kiểm thử tính đúng đắn và nghiệm thu serving

9.1. Các lớp kiểm tra

Kiểm thử M5 cần tách ba câu hỏi: đầu vào có hợp lệ không; dữ liệu đã chuyển vào MongoDB có đúng không; ứng dụng có truy vấn và diễn giải đúng không. Mỗi câu hỏi có bằng chứng khác nhau. CheckOnly kiểm tra nguồn, verifier đối chiếu snapshot trong database, còn unit/integration/UI tests kiểm tra hành vi chương trình và các tình huống lỗi. Health check của container không thể thay thế các phép kiểm tra dữ liệu này.

m5/verify_batch.py lấy snapshot active đã published, đọc từng collection, tính digest và so với giá trị nguồn lưu lúc ingest. Chương trình cũng kiểm tra unique index có đủ khóa và thuộc tính unique. Báo cáo thành công ghi số document, coverage và kết quả các bảng; với bộ mẫu, terminal có dòng *ALL 20 COLLECTIONS CHECKED: PASS; 74921 documents*.

9.2. Tình huống kiểm thử và bằng chứng

Kiểm thử đầu vào cần có schema sai, thiếu bảng, count/coverage sai và khóa trùng. Kiểm thử ingest cần có lần nạp đầu, nạp lại, lỗi giữa chừng, cạnh tranh loader và khôi phục ở ranh giới publish. Kiểm thử snapshot phải chứng minh bộ mới không kế thừa document không còn trong nguồn và nạp bộ cũ không tự rollback. Kiểm thử metrics cần có tỷ lệ trọng số, null, mẫu số bằng 0 và delay âm.

Các bài kiểm tra giao diện phải đi qua trang và bộ lọc thực tế, không chỉ xác nhận app khởi động. Biểu đồ cần được kiểm tra về quần thể donut, tổng treemap, vai trò đi/đến của cùng một sân bay trong Sankey và việc animation không tự tạo số 0 cho nhóm thiếu quan sát. Kiểm tra trên trình duyệt cũng hỗ trợ phát hiện lỗi render hoặc bố cục mà unit test không quan sát được.

Biên bản ngày 07/11/2026 ghi 21 tests passed và một DeprecationWarning từ dependency, đồng thời có kiểm tra dashboard với MongoDB thực, nhánh JSONL/HDFS

cùng fingerprint và stop/start giữ dữ liệu [5]. Đây là evidence của bản code và môi trường đã kiểm tra, không phải chứng nhận rằng mọi môi trường mới hoặc streaming end-to-end đều hoàn tất. Tests dùng database ngẫu nhiên *aero_m5_test_** và dọn sau kiểm thử, không ghi fixture vào database demo.

9.3. Tiêu chí hoàn thành và giới hạn kết luận

Task nghiệm thu hoàn tất khi có log/report truy xuất được đúng phiên bản code và snapshot, các test cần thiết đạt, UI đối chiếu được với dữ liệu và những giới hạn chưa kiểm tra được ghi rõ. Với batch, bằng chứng M5 chứng minh dữ liệu phục vụ khớp nguồn đã inspect lúc ingest. Muốn kết luận KPI đúng từ raw cần kết hợp kiểm chứng công thức và chất lượng dữ liệu của M2; verifier không thay thế công việc đó.

10. Task 9 — Tổ chức vận hành, bảo toàn dữ liệu và bàn giao

10.1. Quy trình vận hành hằng ngày

Một bản code chỉ hoàn thành chức năng kỹ thuật khi người khác có thể vận hành được. Quy trình chuẩn của M5 là nhận input, kiểm tra đường dẫn, start môi trường, CheckOnly, load, verify, mở UI và lưu evidence. Với cùng máy còn volume hợp lệ, lần sử dụng sau chỉ cần start, verify và mở dashboard; không bắt buộc nạp lại bộ nguồn. Scripts PowerShell đóng gói các thao tác Docker và kiểm tra exit code để giảm việc người dùng phải nhớ lệnh dài.

stop_m5.ps1 dừng riêng serving nhưng giữ volume MongoDB. Không dùng tùy chọn xóa volume như một cách dừng thông thường. Với máy mới, MongoDB chưa có dữ liệu dù code giống nhau, nên phải nạp snapshot hoặc thực hiện phương án phục hồi đã kiểm tra. Các báo cáo/log theo tên cố định có thể bị ghi lại ở lần chạy sau; muốn giữ bằng chứng một lần chạy cần chép hoặc đóng gói trước khi tiếp tục.

10.2. Vị trí output và nội dung bàn giao

Kết quả chính nằm trong database *aero_m5*; dashboard mặc định mở ở *http://localhost:8501*. Logs nằm trong *serving-layer/logs*, reports và ảnh được tập hợp trong *serving-layer/evidence*. File tải PNG/CSV từ trình duyệt nằm ở nơi tải về đã chọn, cần chép vào evidence nếu muốn đưa vào bộ bàn giao. Thư mục snapshot M2 là input để tái nạp, không phải output mới được M5 tạo ra.

collect_evidence.ps1 lấy trạng thái container, chạy verify lại, copy logs, tài liệu và các file report/ảnh/CSV/TXT phù hợp ở cấp gốc evidence, rồi tạo *M5_HANDOFF_<ngày_giờ>.zip*. Script không tự chụp ảnh màn hình và ZIP này không chứa MongoDB volume, toàn bộ code hoặc snapshot nguồn. Vì vậy, bàn giao để thành viên tái lập cần đủ code, dữ liệu M2, evidence và ghi chú môi trường.

Trong mô hình demo lịch sử, một phương án phục hồi đơn giản là dựng MongoDB trống và nạp lại snapshot M2. Khi vận hành có thêm dữ liệu streaming hoặc metadata quan trọng, cần chính sách backup/restore riêng và thử phục hồi trên môi trường phù hợp. Giữ volume chỉ giúp dữ liệu tồn tại qua vòng đời container, không tự thay thế sao lưu trước sự cố lưu trữ.

10.3. Kết quả cần đạt

Thành viên nhận bàn giao phải biết input ở đâu, chạy phần nào trước, kết quả PASS trông như thế nào và phải xem log nào khi lỗi. Họ cũng phải hiểu localhost là máy hiện tại; gửi link localhost không làm người khác xem được dashboard đang chạy trên máy bạn. Nếu mục tiêu là dùng chung, cần một phương án triển khai mạng được thống nhất thay vì chỉ sao chép URL.

11. Task 10 — Thiết kế phương án chuyển hạ tầng sang Kubernetes

11.1. Mục tiêu và phạm vi thay đổi

Kubernetes là phương án điều phối container ở mốc triển khai tiếp theo. Việc chuyển đổi có thể giữ phần lớn code Python và Dockerfile; thay đổi chủ yếu nằm ở cách chạy image, quản lý cấu hình, mạng và lưu trữ. Đây là chuyển từ Compose sang Kubernetes cho phần điều phối, không phải loại bỏ container hoặc buộc viết lại dashboard. Trong repo hiện tại, cấu hình và bằng chứng triển khai Kubernetes chưa được bổ sung.

Thiết kế đề xuất dùng Deployment và Service cho dashboard, StatefulSet và Service cho MongoDB, PersistentVolumeClaim cho lưu trữ database, và Job cho loader/verifier. Các loại workload được chọn theo việc tiến trình chạy thường trực, cần định danh/lưu trữ ổn định hay thực hiện đến khi hoàn tất [9]. Cấu hình thường có thể đưa vào ConfigMap; URI có thông tin đăng nhập cần cơ chế quản lý Secret phù hợp môi trường.

11.2. Những nội dung phải triển khai bổ sung

Cluster phải truy cập được image ứng dụng từ registry hoặc qua cách nạp image phù hợp với môi trường local. Cần chọn storage backend và chính sách cấp/giữ volume; volume Docker hiện tại không tự chuyển thành PVC. Snapshot JSONL phải được đưa tới nơi Job đọc được, còn logs/report phải được lấy ra hoặc lưu vào nơi bền vững. Job cũng cần xử lý phụ thuộc MongoDB sẵn sàng thay vì giả định có thể giữ nguyên depends_on của Compose.

Mạng Compose *batch-layer_default* không tự nối tới Kubernetes. Nếu đọc HDFS trực tiếp trong khi M2 còn chạy ở Compose, phải giải quyết kết nối tới cả NameNode và DataNode. Đối với Streamlit, bắt đầu một replica giúp đơn giản hóa triển khai; khi dùng nhiều replica cần xem xét session affinity theo đặc điểm client–server và trạng thái phiên [8]. Một MongoDB instance có PVC cũng chưa tạo thành replica set dự phòng.

11.3. Lộ trình và tiêu chí hoàn thành

Lộ trình phù hợp là giữ hệ thống Compose đang hoạt động, dựng riêng M5 trên cluster với storage mới, đưa snapshot JSONL vào, load và verify, đối chiếu UI, rồi thử tạo lại Pod và kiểm tra persistence. Sau khi luồng batch ổn định mới nối HDFS trực tiếp và streaming. Cách làm này chia bài toán hạ tầng thành các mốc có thể quan sát và phục hồi, đồng thời giữ nguồn để đối chiếu.

Task này chỉ hoàn tất khi có manifests, môi trường chạy thật, cấu hình image/storage/network/probes/resources và evidence triển khai tương ứng. Mô tả kiến trúc hoặc có thể bật Kubernetes trên máy chưa phải bằng chứng đã chuyển project thành công. Nếu yêu cầu môn học cần cloud hoặc cụm nhiều máy, phải kiểm tra riêng các yêu cầu đó; một cluster local không tự chứng minh tính sẵn sàng cao.

12. Tổng kết quan hệ giữa các nhiệm vụ

Mười nhiệm vụ của M5 tạo thành một chuỗi phụ thuộc. Hợp đồng xác định ý nghĩa dữ liệu; mô hình MongoDB bảo đảm khóa và phiên bản; hạ tầng tạo môi trường chạy; loader biến input thành bộ phục vụ; truy vấn và dashboard biến bộ phục vụ thành câu trả lời cho người dùng. Streaming bổ sung cơ chế cập nhật liên tục, kiểm thử đánh giá tính đúng đắn, còn vận hành và triển khai bảo đảm kết quả có thể bàn giao và duy trì.

Đối với project hiện tại, phạm vi đã có bằng chứng là batch lịch sử M2, hai nhánh nhận JSONL/aggregate HDFS, MongoDB, dashboard và kiểm thử local. Streaming có writer cùng contract đề xuất nhưng còn chờ M4; Kubernetes là nhiệm vụ triển khai tiếp theo. Việc phân biệt trạng thái này giúp team đánh giá đúng tiến độ và phân công phần còn lại mà không nhầm bản thiết kế với hệ thống đã nghiệm thu.

Tài liệu tham khảo

[1] Nhóm dự án AERO. *Big_Data_Project_Plan_5_Members.pdf*, trang 6–8. Tài liệu phân công M4, M5 và trách nhiệm tích hợp trong workspace.

[2] Nhóm dự án AERO. *HUONG_DAN_BAN_GIAO_M2_M5.md* và manifest của snapshot *20261107T042703414867Z*. Thư mục *batch-layer/doc_guide* và *batch-layer/spark_jobs/export/m2_to_m5*.

[3] Nhóm dự án AERO. *M4_TO_M5.md: Contract đề xuất M4 → M5*. Thư mục *serving-layer/contracts*; cần xác nhận với output M4 thực tế.

[4] Nhóm dự án AERO. Mã nguồn *serving-layer*: Compose, Dockerfiles, scripts và các module *contracts*, *batch_writer*, *queries*, *metrics*, *charts*, *stream_writer*. Đối chiếu ngày 07/11/2026.

[5] Nhóm dự án AERO. *KET_QUA_KIEM_THU_M5.md*. Biên bản local ngày 07/11/2026, thư mục *serving-layer/doc_guide*.

[6] MongoDB. Unique Indexes and Duplicate Values. Tài liệu chính thức, truy cập ngày 07/11/2026.

[7] MongoDB. Bulk Write Operations — PyMongo. Tài liệu chính thức, truy cập ngày 07/11/2026.

[8] Streamlit. Understanding Streamlit's client–server architecture. Tài liệu chính thức, truy cập ngày 07/11/2026.

[9] Kubernetes. Workloads. Tài liệu chính thức, truy cập ngày 07/11/2026.

Phụ lục A. Liên hệ giữa nhiệm vụ và thao tác thực hành

Phần chính trình bày theo bài học và nhiệm vụ thiết kế. Phụ lục này cung cấp chuỗi lệnh đại diện cho luồng batch đã có code, để người đọc liên hệ lý thuyết với chương trình thực tế. Các lệnh dưới chạy trong PowerShell Windows, Docker Desktop đang hoạt động. Thay đường dẫn nếu máy nhận lưu dự án ở vị trí khác. Hướng dẫn thao tác và xử lý lỗi chi tiết đã có trong *HUONG_DAN_CHAY_TU_DAU_DEN_OUTPUT_M5.md*.

Khai báo đường dẫn và chọn snapshot chứa trực tiếp manifest cùng các JSONL. Tên biến PowerShell phải có dấu *$*; nếu mở terminal mới thì khai báo lại. Khi nhận bộ ZIP M2, giải nén và có thể thay *AeroSnapshot* bằng đường dẫn thư mục data bên trong bộ bàn giao.

```
$AeroProject = 'F:\Project-BigData-Storage-and-Processing'
$AeroServing = Join-Path $AeroProject 'serving-layer'
$AeroExport = Join-Path $AeroProject 'batch-layer\spark_jobs\export'
$AeroSnapshot = Join-Path $AeroExport 'm2_to_m5\20261107T042703414867Z'
Set-Location $AeroServing
Test-Path .\compose.yml
Test-Path (Join-Path $AeroSnapshot 'manifest.json')
```

Hai phép Test-Path cần trả True. Sau đó khởi động hạ tầng và kiểm tra dữ liệu nguồn trước khi nạp thật. Chạy từng lệnh và chờ kết thúc; nếu lỗi, đọc thông báo và log trước khi sang lệnh tiếp theo. CheckOnly đạt chưa có nghĩa dữ liệu đã vào database.

```
.\scripts\start_m5.ps1
.\scripts\load_m2.ps1 -SnapshotPath $AeroSnapshot -CheckOnly
.\scripts\load_m2.ps1 -SnapshotPath $AeroSnapshot
.\scripts\verify_m5.ps1
Start-Process 'http://localhost:8501'
```

Với bộ mẫu, verifier cần báo PASS cho 20 collection và 74.921 document. Để xem trực tiếp, mở Compass, tạo kết nối *mongodb://localhost:27017*, chọn database *aero_m5*, đọc snapshot_id trong *serving_state* rồi dùng giá trị đó lọc collection KPI. MongoDB server phải đang chạy; cấu hình local hiện tại chưa yêu cầu tài khoản/mật khẩu.

Nếu dùng shell, dòng sau chạy trong PowerShell để mở mongosh bên trong container. Chỉ sau khi dấu nhắc chuyển sang mongosh mới nhập đoạn JavaScript tiếp theo; không dán JavaScript hoặc Python trực tiếp vào dấu nhắc PowerShell.

```
docker compose -f compose.yml exec mongo mongosh aero_m5
```

```
var sid = db.serving_state.findOne({_id: 'historical'}).snapshot_id;
db.kpi_month_overall.countDocuments({snapshot_id: sid});
db.kpi_month_overall.findOne({snapshot_id: sid, year: 2025, month: 1});
```

Với snapshot tham chiếu, phép đếm trả 51 document KPI tháng. Gõ *exit* để về PowerShell. Trước khi bàn giao, có thể chạy tests, lưu ảnh thực tế trực tiếp vào evidence và đóng gói; script đóng gói sẽ verify lại. Khi kết thúc, dừng bằng script để giữ volume. ZIP evidence phải được gửi kèm code và snapshot nguồn nếu người nhận cần tái lập trên máy khác.

```
.\scripts\test_m5.ps1
.\scripts\collect_evidence.ps1
.\scripts\stop_m5.ps1
```

Nếu *.ps1* bị Execution Policy chặn, dùng các lệnh Docker tương đương trong phụ lục của hướng dẫn thực hành; không cần thay đổi policy vĩnh viễn. Các lỗi đường dẫn, cổng, Docker daemon hoặc thiếu snapshot cần được sửa trước khi diễn giải là lỗi thuật toán. Phụ lục này mô tả thao tác, không xác nhận các lệnh vừa được chạy lại khi biên soạn tài liệu.